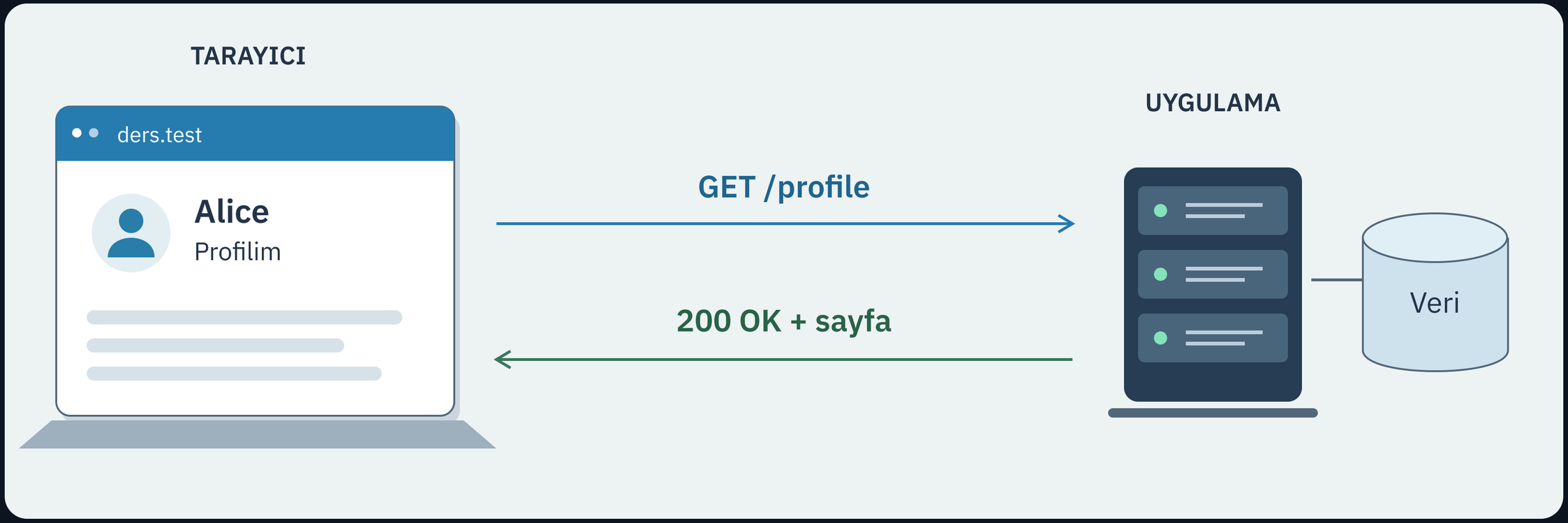
2026-27 GÜZ · HAFTA 3

Tarayıcı güvenliği

HTTP, oturumlar ve istemci tarafı saldırılar



BUGÜN

Önce normal akış, sonra saldırı

1 · Hatırlatma

SQLi
Command injection
Path traversal

→

2 · Normal akış

HTTP
Cookie ve session
Kimlik doğrulama

→

3 · Saldırılar

XSS
CSRF
Clickjacking

Her örnekte: ne değişti, ne oldu, nasıl düzelir?

Üç hata, üç farklı kontrol

SQLi

1 OR 1=1

Sabit sorgu
+ ayrı parametre

COMMAND INJECTION

alice; echo DEMO

Shell yok
+ argüman listesi

PATH TRAVERSAL

../private/file

Yolu çöz
+ izinli dizini kontrol et

SQLi kapandı. Bu profil kimin?

Parametreli sorgu

```
1  db.execute("SELECT name FROM users WHERE id = ?", (uid,))
```

Alice

`GET /profile?id=2`

→

Sunucu

Sorgu güvenli

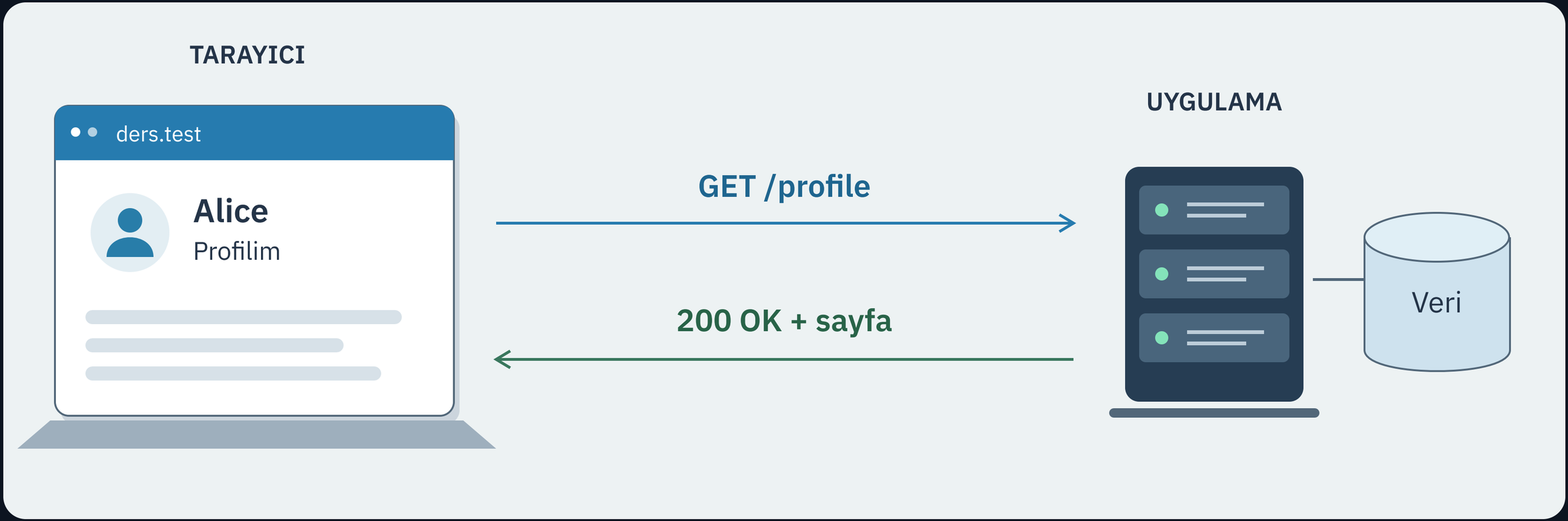
→

Bob'un özel profili

Alice görebilmeli mi?

İSTEMCİ / SUNUCU

Client ister, sunucu yanıtlar



Client; tarayıcı, mobil uygulama veya curl olabilir.

İSTEMCİYE GÜVENME

Buton gizli olsa da istek gönderilebilir

ders.test/users

Sil butonu yok

Alice normal kullanıcı.

Şematik tarayıcı görünümü

Arayüz dışında gönderilen istek

```
1 DELETE /api/users/2 HTTP/1.1
2 Host: ders.test
```

İzin kontrolü sunucuda yapılır.

Adresin hangi kısmı nereye gider?

https://	ders.test	:443	/profile	?id=1	#details
scheme	host	port	path	query	fragment

Sunucuya giden yol ve query

`/profile?id=1`

Tarayıcıda kalan fragment

`#details`

HTTP · REQUEST

Bir giriş isteği

request.http

```
POST /login HTTP/1.1
Host: ders.test
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

username=alice&password=lab
```

1 · İstek satırı

POST /login

2 · Header'lar

Host, içerik türü, uzunluk

3 · Boş satırdan sonra

Body: form değerleri

HTTP · RESPONSE

Yanıt: durum kodu + header + body

response.http

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{"name":"alice"}
```

200 · Başarılı

Profil verisi döner.

401 · Geçerli kimlik bilgisi yok

Kimlik doğrulaması gerekir.

403 · İzin yok

İşlem yapılmaz.

GET okur; POST işlem başlatır

Profili getir

```
GET /profile
```

Temayı değiştir

```
POST /settings/theme

theme=light
```

POST kullanmak tek başına CSRF koruması değildir.

HTTPS

Ağdaki aktarımı TLS korur



DevTools içeriği gösterir; tarayıcı bağlantının ucudur.

HTTP · STATELESS

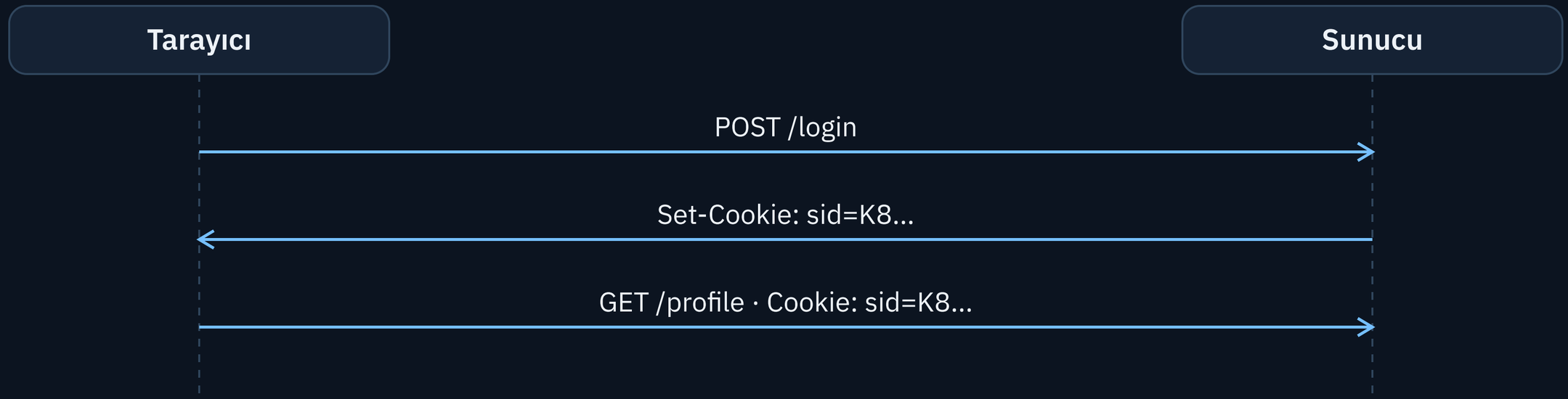
İkinci isteğin Alice’ten geldiğini nasıl anlarız?



HTTP, önceki giriş bilgisini kendiliğinden hatırlamaz.

COOKIE

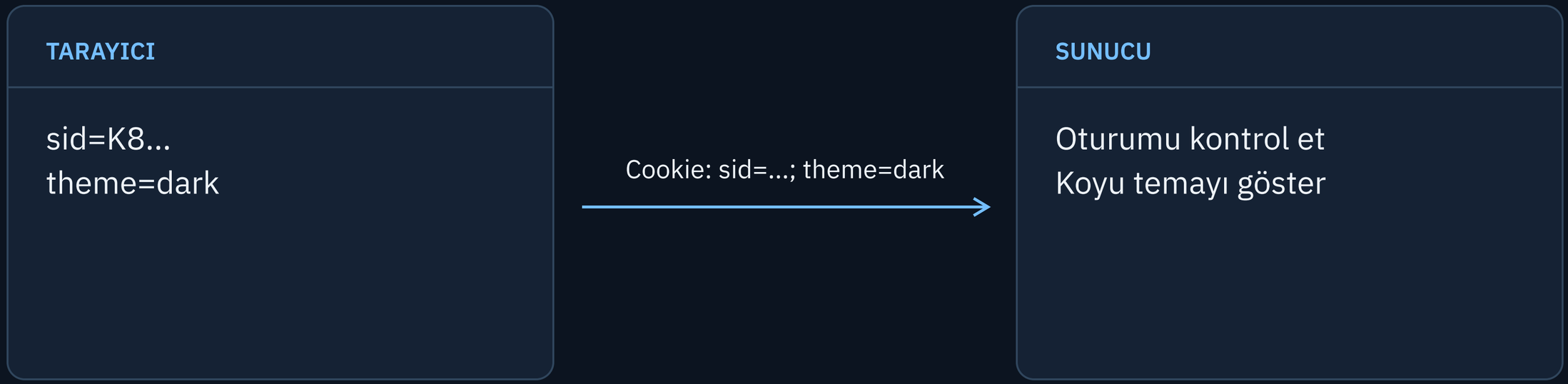
Yanıtta kaydet, sonraki istekte gönder



K8… gösterim için kısaltılmış rastgele oturum kimliği.

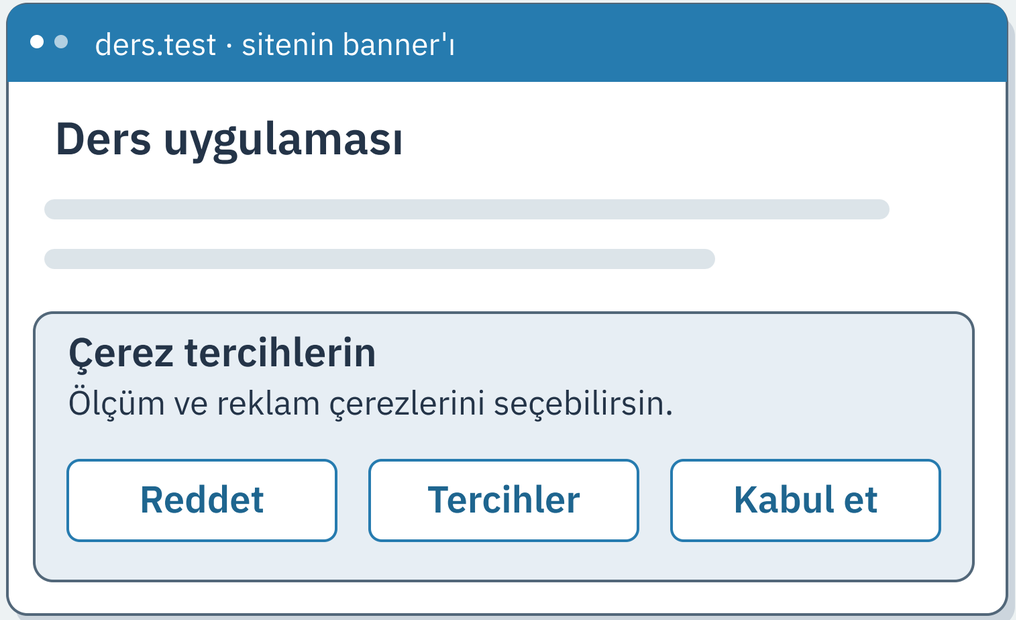
COOKIE

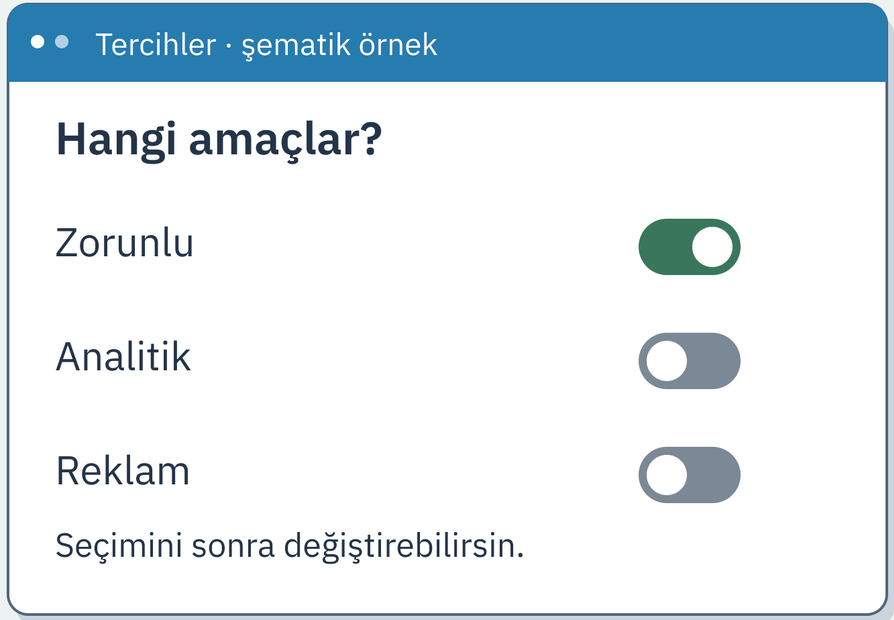
Cookie yalnızca oturum için kullanılmaz



ÇEREZ İZİNLERİ

“Kabul et” dediğimizde neyi seçiyoruz?

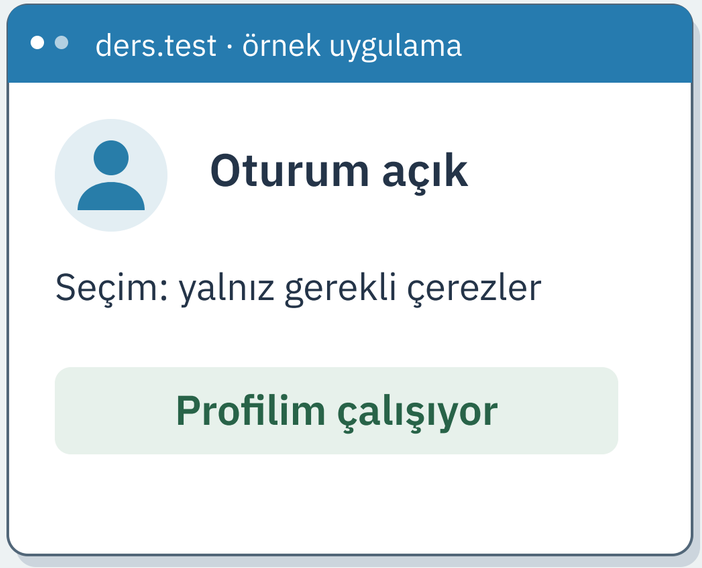




Bu örnekte analitik ve reklam isteğe bağlı; seçimden önce kapalı.

ÇEREZ İZİNLERİ

“Reddet” demek, oturumu kapatmak değildir



BU ÖRNEKTE TARAYICIDA

sid=K8…	Giriş için gerekli
consent=necessary	Tercihi hatırlar
ad_id=T42	Reklam çerezi yok

Reddet: bu sitenin isteğe bağlı çerezlerini reddetmek. Tüm çerezleri silmek değil.

ÜÇÜNCÜ TARAF ÇEREZİ

İki farklı sitede aynı reklam hizmeti



Tarayıcı bu kullanımı engelleyebilir veya siteye göre ayırabilir.

İKİ AYRI KONTROL

Siteye izin verdim; tarayıcı yine engelleyebilir



Sitenin banner'ı tarayıcının güvenlik ayarını değiştirmez.

Üç ayar, üç farklı görev

Secure

HTTPS ile gönder

Ağdaki aktarım

HttpOnly

JavaScript okumasın

Cookie'ye script erişimi

SameSite

Site dışını sınırla

İsteğe cookie eklenmesi

HTTPS yanıtı

```
1 Set-Cookie: sid=K8...; Secure; HttpOnly; SameSite=Lax; Path=/
```

COOKIE · KAPSAM

Path gönderimi daraltır, uygulamaları ayırmaz

ders.test yanıtı

```
1 Set-Cookie: theme=dark; Path=/course
```

ders.test/course

Cookie gönderilir

Path eşleşiyor.

Şematik tarayıcı görünümü

ders.test/shop

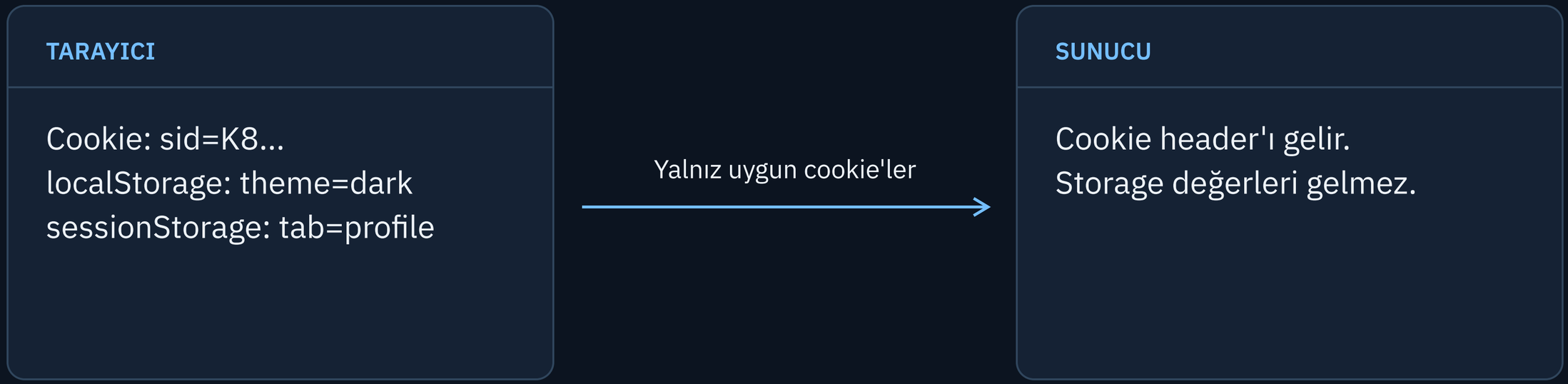
Bu isteğe eklenmez

Path eşleşmiyor.

Şematik tarayıcı görünümü

TARAYICIDA SAKLAMA

Storage, cookie gibi otomatik gönderilmez



sessionStorage, sunucudaki session değildir.

TARAYICIDA SAKLAMA

Tercih saklamak, giriş yapmak değildir

Tarayıcı · Console

```
1  localStorage.setItem("theme", "dark");
2  sessionStorage.setItem("activeTab", "profile");
```

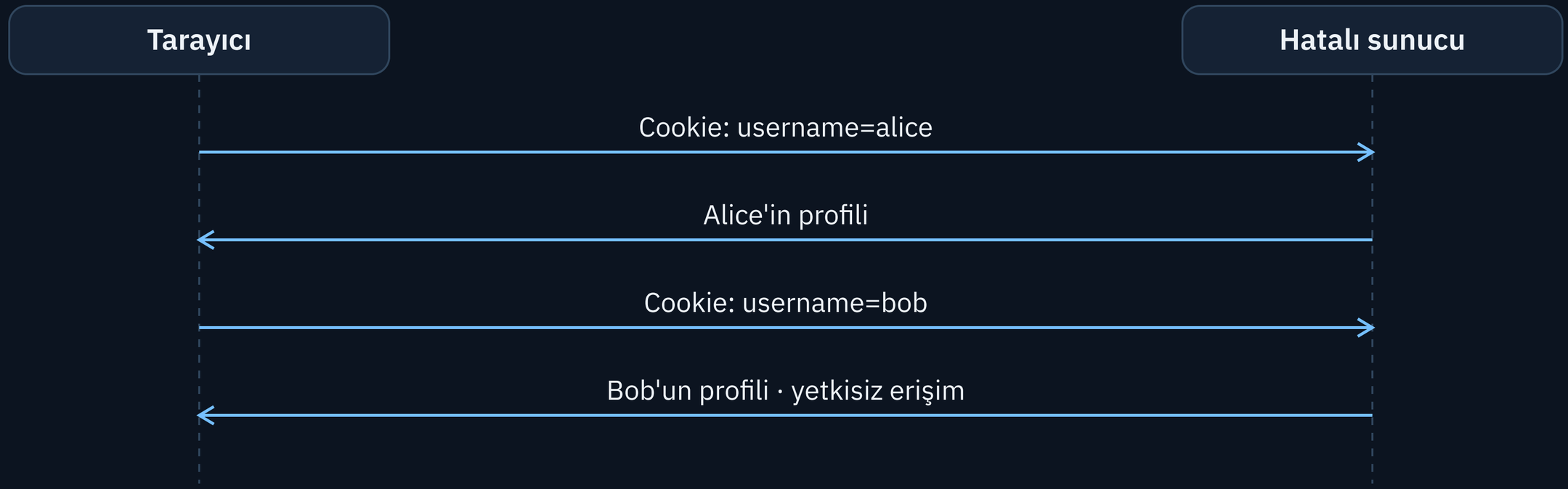
ders.test/profile

Koyu tema · Profil sekmesi

Sunucu, bu değerlerle kullanıcı kimliği doğrulamaz.

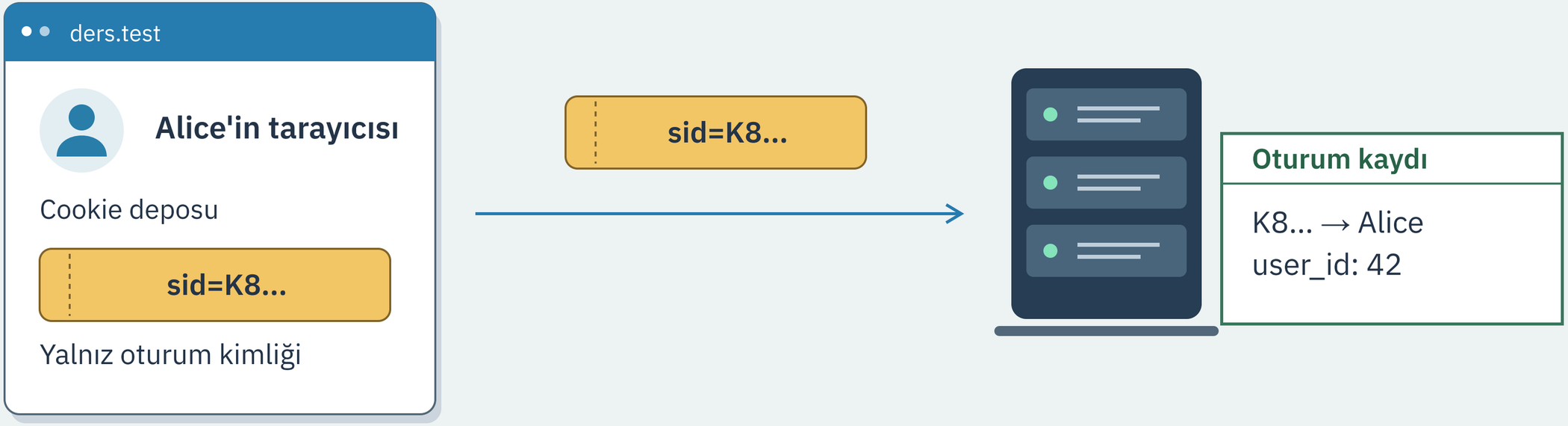
Şematik tarayıcı görünümü

Cookie’ye username=alice yazsak?



SESSION · SUNUCUDA SAKLAMA

Tarayıcı kimliği taşır, kayıt sunucudadır



Kimlik tarayıcıda; oturum kaydı sunucuda.

Oturum verisi imzalı cookie’de taşınabilir



Veri tarayıcıda; doğrulama sunucuda.

Kavramsal gösterim; gerçek cookie biçimi değil.

İMZALI COOKIE

Okumak başka, değiştirmek başka

Orijinal

```
user_id=42 + geçerli imza
→ doğrulama başarılı
```

Değiştirilmiş kopya

```
user_id=7 + eski imza
→ doğrulama başarısız
```

İmza veriyi gizlemez; değişikliği tespit eder.

SESSION · İKİ MODEL

Cookie var. Peki içinde ne var?

SUNUCUDA OTURUM

Rastgele kimlik

sid=K8...
→ sunucudaki kaydı bul

İSTEMCİDE OTURUM

Veri + imza

user_id=42 + imza
→ imzayı doğrula

Cookie bir taşıma mekanizmasıdır; tek başına session modelini söylemez.

PYTHON · FLASK · COOKIE

Bir yanıtta yaz, sonraki istekte oku

Tema seçimini kaydeden view fonksiyonu

```
response = make_response("Tema kaydedildi")
response.set_cookie("theme", "dark", max_age=86400,
                    secure=True, httponly=True, samesite="Lax")
return response
```

Sonraki isteği işleyen view fonksiyonu

```
theme = request.cookies.get("theme", "light")
```

Cookie: theme=dark · Tarayıcıda saklanır; kullanıcı değiştirebilir.

Aynı işlem: Response yazar, Request okur

Controller · tema seçimini kaydeden action

```
Response.Cookies.Append("theme", "dark", new CookieOptions {
    Secure = true, HttpOnly = true,
    SameSite = SameSiteMode.Lax, MaxAge = TimeSpan.FromDays(1)
});
```

Controller · sonraki isteği işleyen action

```
var theme = Request.Cookies["theme"] ?? "light";
```

Cookie yoksa light. Bu değer kullanıcı kimliği değildir.

PYTHON · FLASK · SESSION

Önce imza anahtarını ve cookie ayarlarını ver

app.py · başlangıç

```
app = Flask(__name__)
app.config.update(
    SECRET_KEY=os.environ["FLASK_SECRET_KEY"],
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
)
```

Güçlü, gizli anahtar ortamdan gelir. Uygulama HTTPS üzerinden çalışır.

PYTHON · FLASK · SESSION

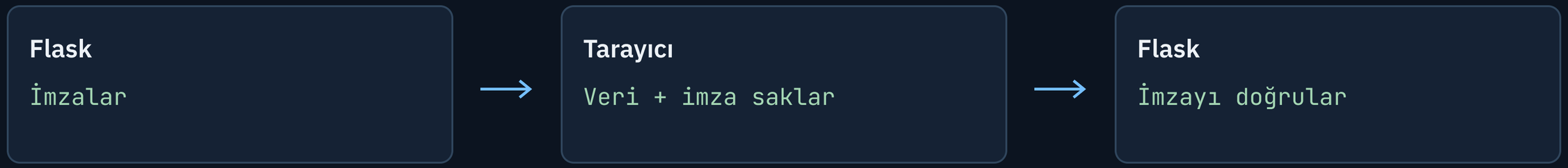
Sözlüğe yazdık; veri imzalı cookie'ye gitti

Tema seçimini kaydeden view · yaz

```python
session["theme"] = "dark"
```

Sonraki view · oku

```python
theme = session.get("theme", "light")
```



İmzalı ≠ şifreli. sessionStorage ile ilgisi yok.

C# · ASP.NET CORE · SESSION

Oturum için sunucuda bir depo aç

Program.cs · session kurulumundan kesit

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options ⇒ {
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.HttpOnly = true;
    options.Cookie.SameSite = SameSiteMode.Lax;
});
var app = builder.Build();
app.UseSession();
```

Bu örnekte depo uygulamanın belleği. Cookie yalnız oturum kimliğini taşır.

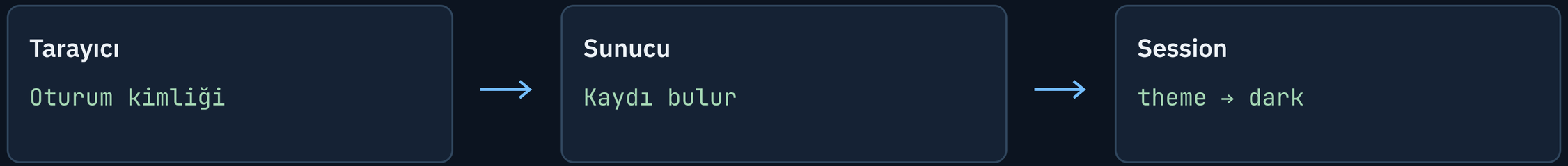
Sözlüğe yazdık; veri sunucudaki depoda kaldı

Controller · tema seçimini kaydeden action

```
1  HttpContext.Session.SetString("theme", "dark");
```

Controller · sonraki action

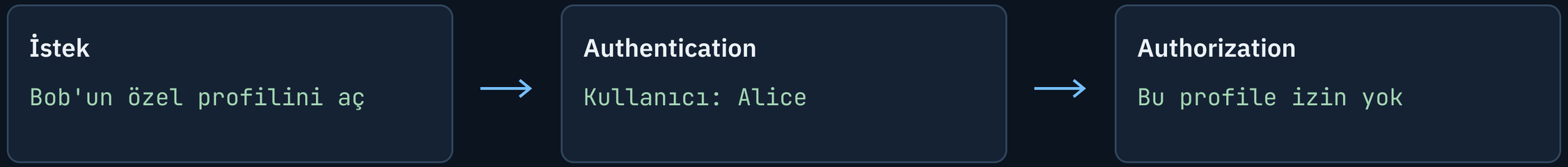
```
1  var theme = HttpContext.Session.GetString("theme") ?? "light";
```



ASP.NET Core Session, kullanıcıyı authenticate etmez.

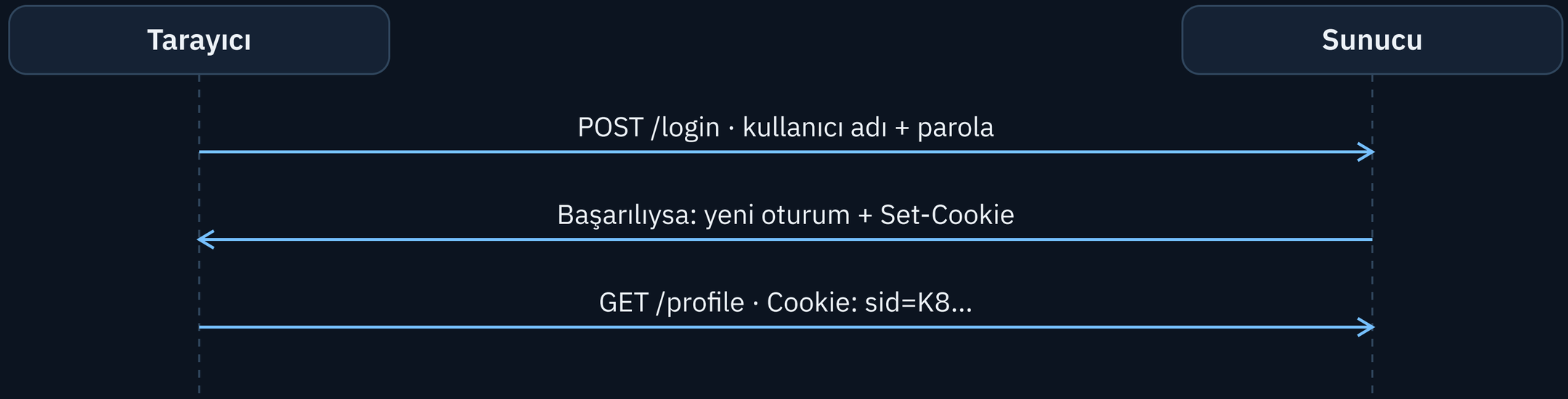
AUTHENTICATION / AUTHORIZATION

Kim olduğun ve ne yapabileceğin ayrı sorular



Giriş yapmak, her kayda erişim hakkı vermez.

Parolayı doğrula, yeni oturum oluştur



Parolanın kendisi cookie'ye yazılmaz.

OTURUMLU İSTEK

Cookie geldi diye doğrudan güvenmeyiz

Kısaltılmış istek

```
1  GET /profile HTTP/1.1
2  Cookie: sid=K8...
```

1 · Oturum

Geçerli mi?

→

2 · Kullanıcı

Kim?

→

3 · İzin

Bu kaynağa erişebilir mi?

OTURUM ÖMRÜ

İki ayrı saat çalışır

TARAYICIDA

Cookie ne kadar kalır?

Max-Age / Expires

SUNUCUDA

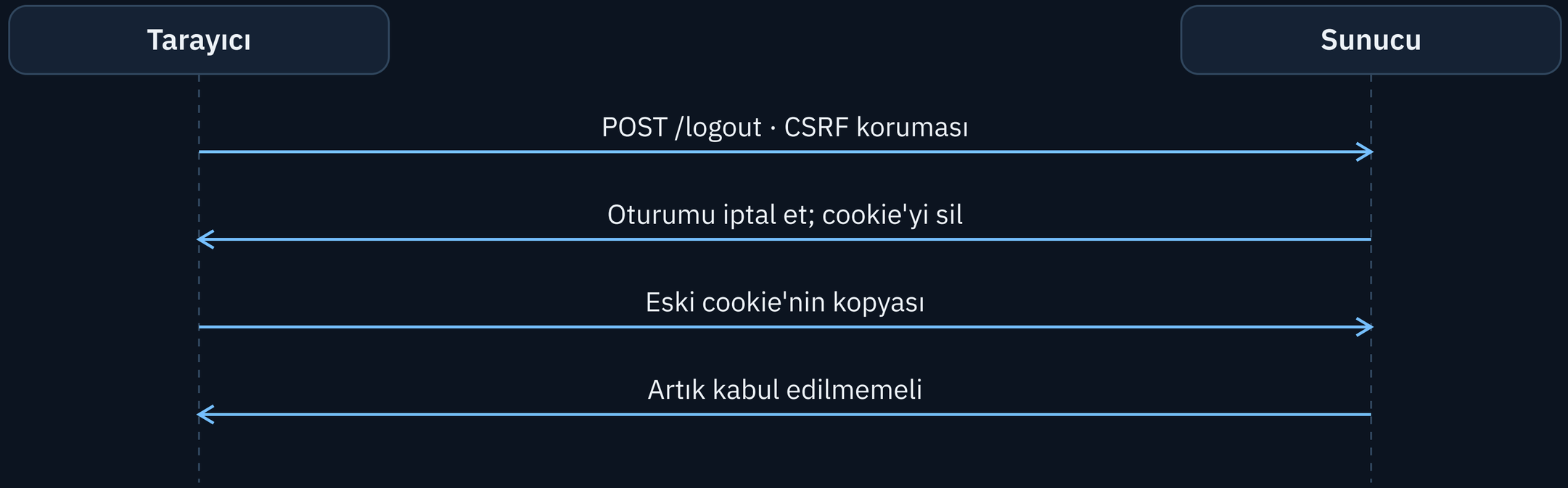
Oturum hâlâ geçerli mi?

Süre / iptal kontrolü

Tarayıcıyı kapatmak güvenilir bir çıkış işlemi değildir.

ÇIKIŞ

Tarayıcıdan sil; sunucuda geçersiz kıl



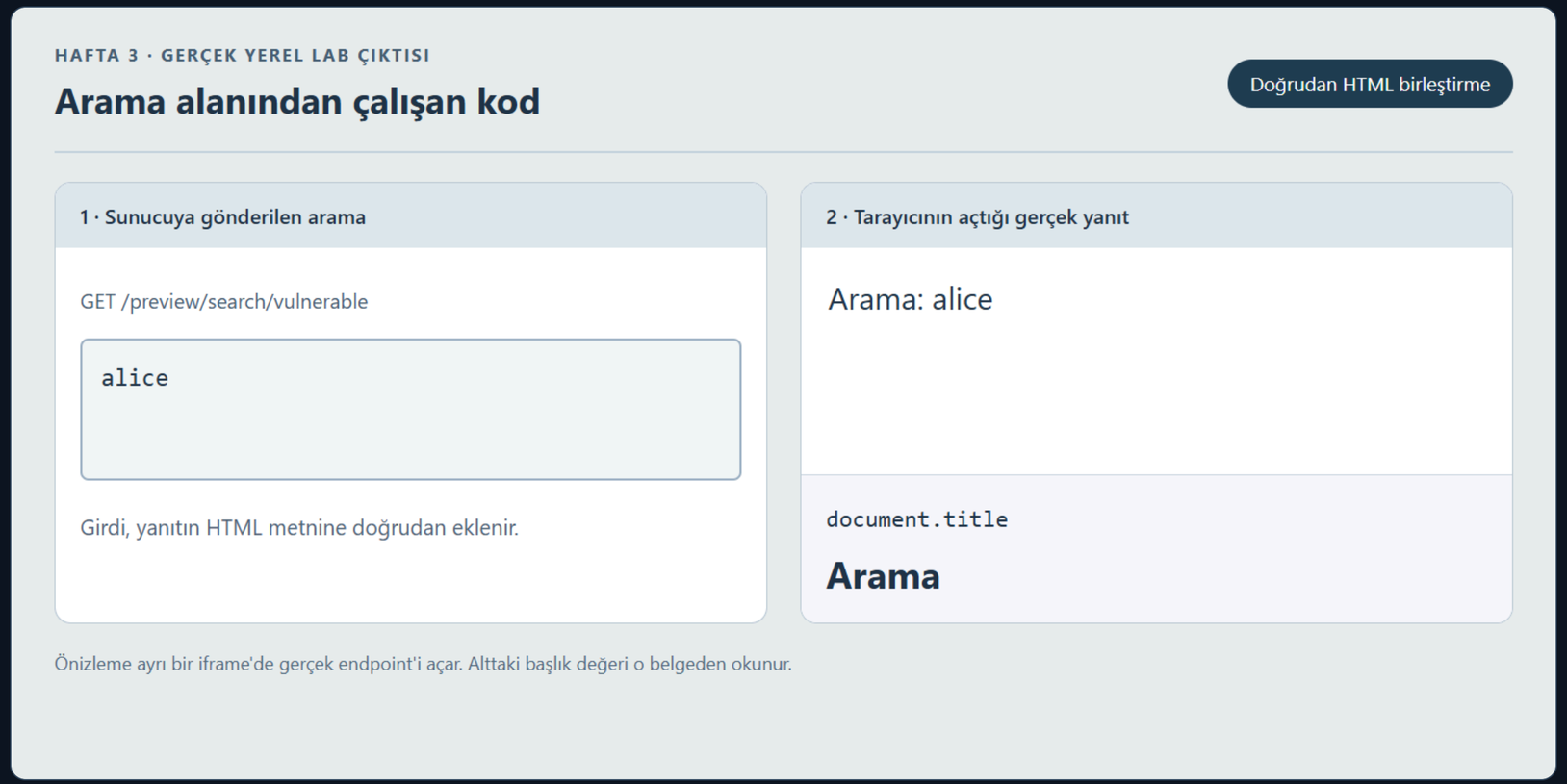
Metin diye aldık, tarayıcı kod diye çalıştırdı



Hafta 2'de SQL ve shell; burada HTML ve JavaScript.

XSS · LAB · NORMAL GİRDİ

Önce alice aratalım



Gerçek lab: arama metni görünür, belge başlığı Arama kalır.

XSS · REFLECTED

Arama metni HTML’e doğrudan ekleniyor

app.py · zafiyetli

```
1  term = request.args.get("q", "")
2  return "<p>Arama: " + term + "</p>"
```

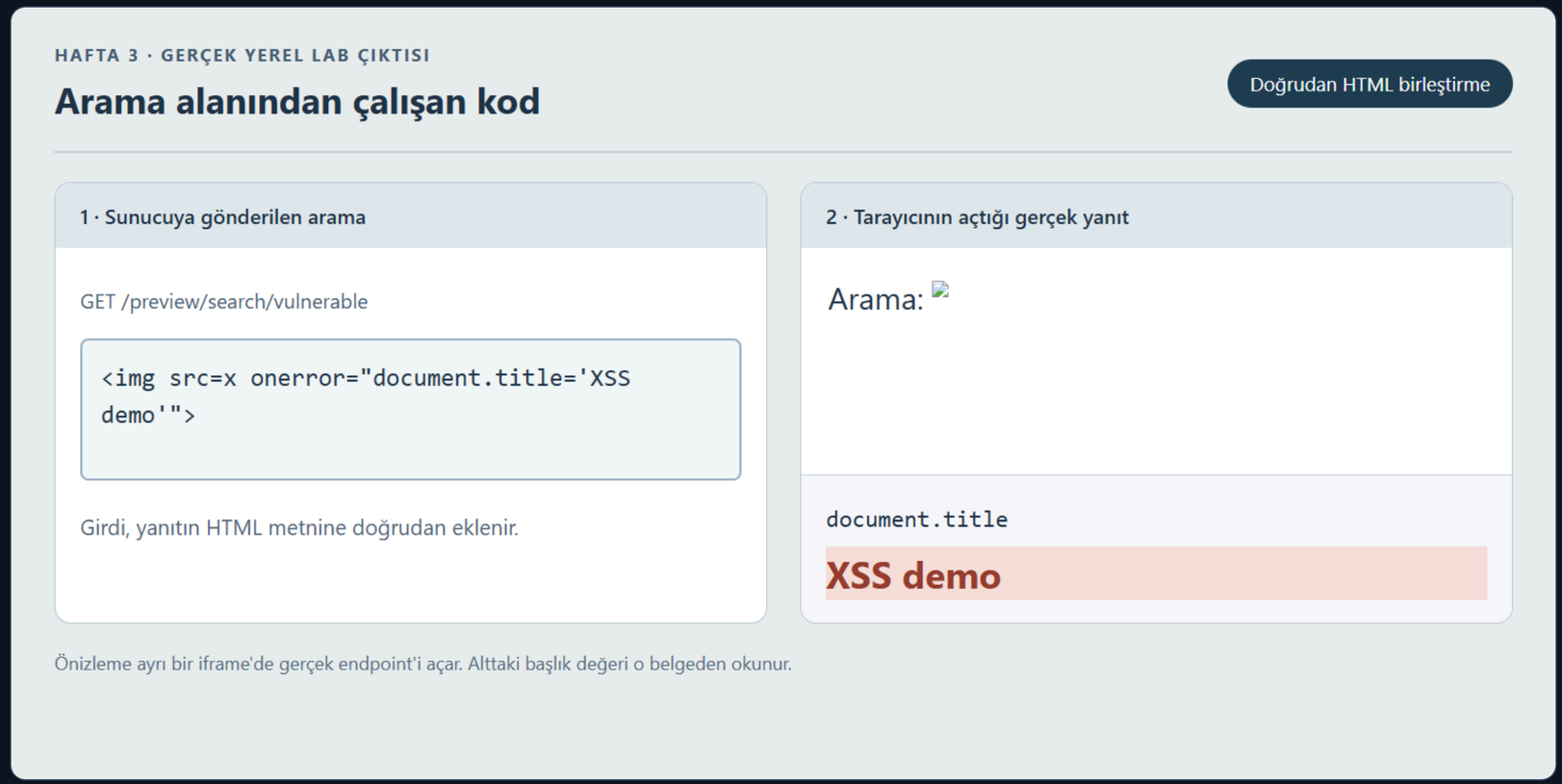
q değeri · yalnız yerel ders örneği

```
1  <img src=x onerror="document.title='XSS demo'">
```

Beklenen

Arama metni

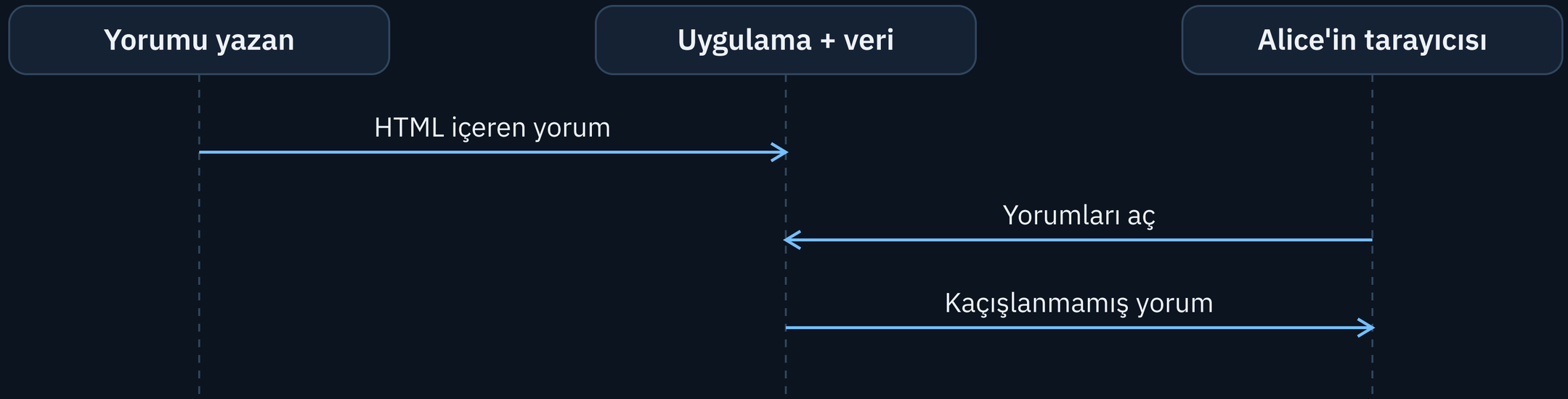
Yalnızca arama yaptık; sayfanın başlığı değişti



Gerçek lab: onerror çalıştı; document.title artık XSS demo.

XSS · STORED

Yorum kaydedilir; sonraki ziyaretçide çalışır



Veritabanına kaydetmek, girdiyi güvenilir yapmaz.

XSS · DÜZELTME

Metni HTML olarak ayrıştırma

app.py · sabit şablona veri gönder

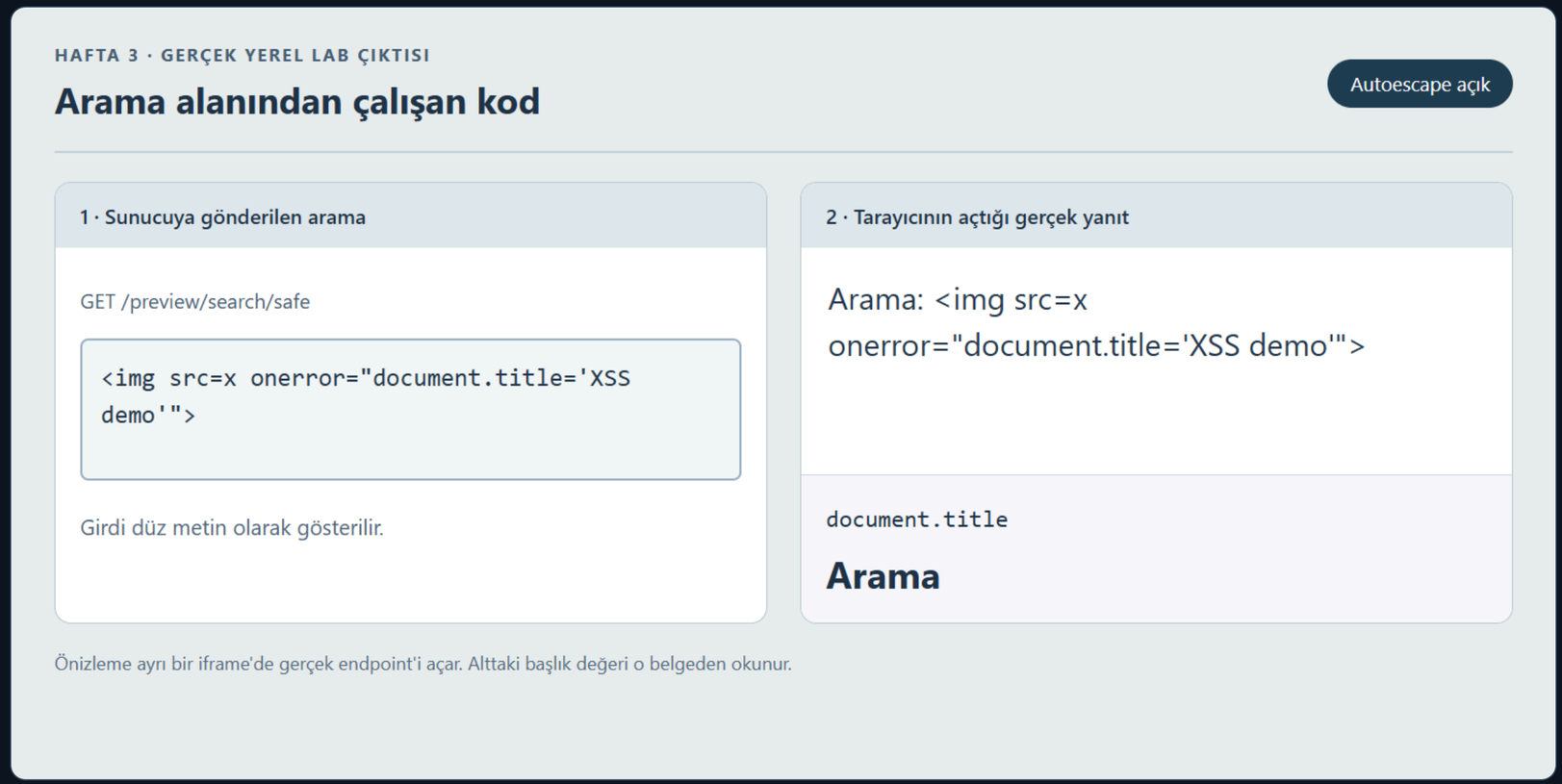
```
1  return render_template("search.html", term=term)
```

search.html · autoescape açık

```
1  <p>Arama: {{ term }}</p>
```

Şablon sabit; arama metni veri olarak eklenir.

Aynı girdi bu kez metin olarak görünüyor



Gerçek lab: HTML kaçışlama açık; img oluşmadı, başlık değişmedi.

HTML kaçışlama, URL'yi güvenli yapmaz

Autoescape açık şablon

```
<a href="{{ url }}">Profil</a>
```

İzin verilmemesi gereken URL scheme'i

```
javascript:alert("demo")
```

URL için ayrıca izin verilen scheme'leri doğrula.

XSS · CSP

Nonce eşleşen script çalışır

HTTP response header

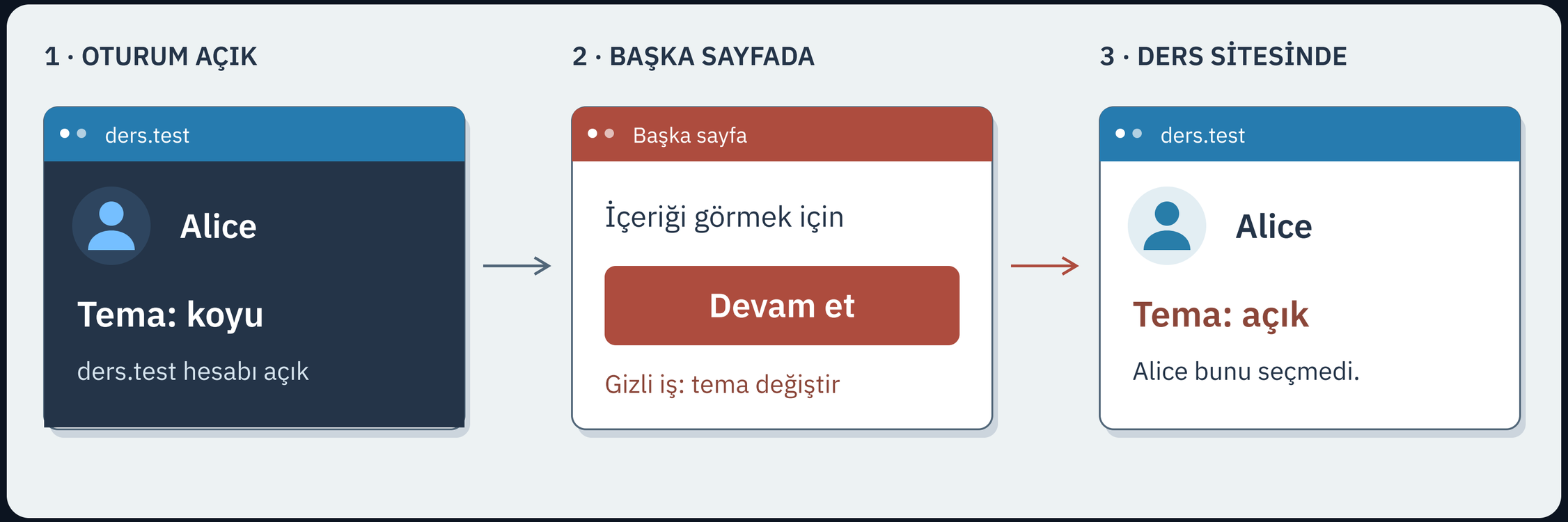
```
Content-Security-Policy: script-src 'nonce-DEMO_NONCE'; object-src 'none'
```

İzin verilen script

```
<script nonce="DEMO_NONCE" src="/app.js"></script>
```

Inline onerror engellenir. CSP, çıktı düzeltmesinin yerine geçmez.

Alice değiştirmedi. Teması yine de değişti.



Başka sayfa, Alice'in açık oturumuyla işlem yaptırıyor. Buna CSRF diyoruz.

Saldırganın parolayı bilmesi gerekmedi

BAŞKA SAYFANIN İSTEĞİ

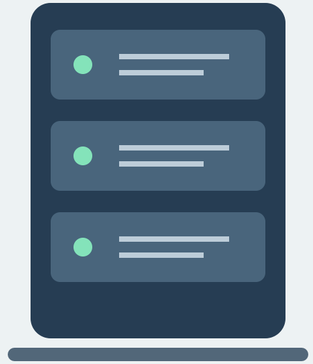
Ders sitesine: temayı açığa al

theme=light

sid=K8...

Tarayıcı ekler
Saldırgan bilmez

Gönder



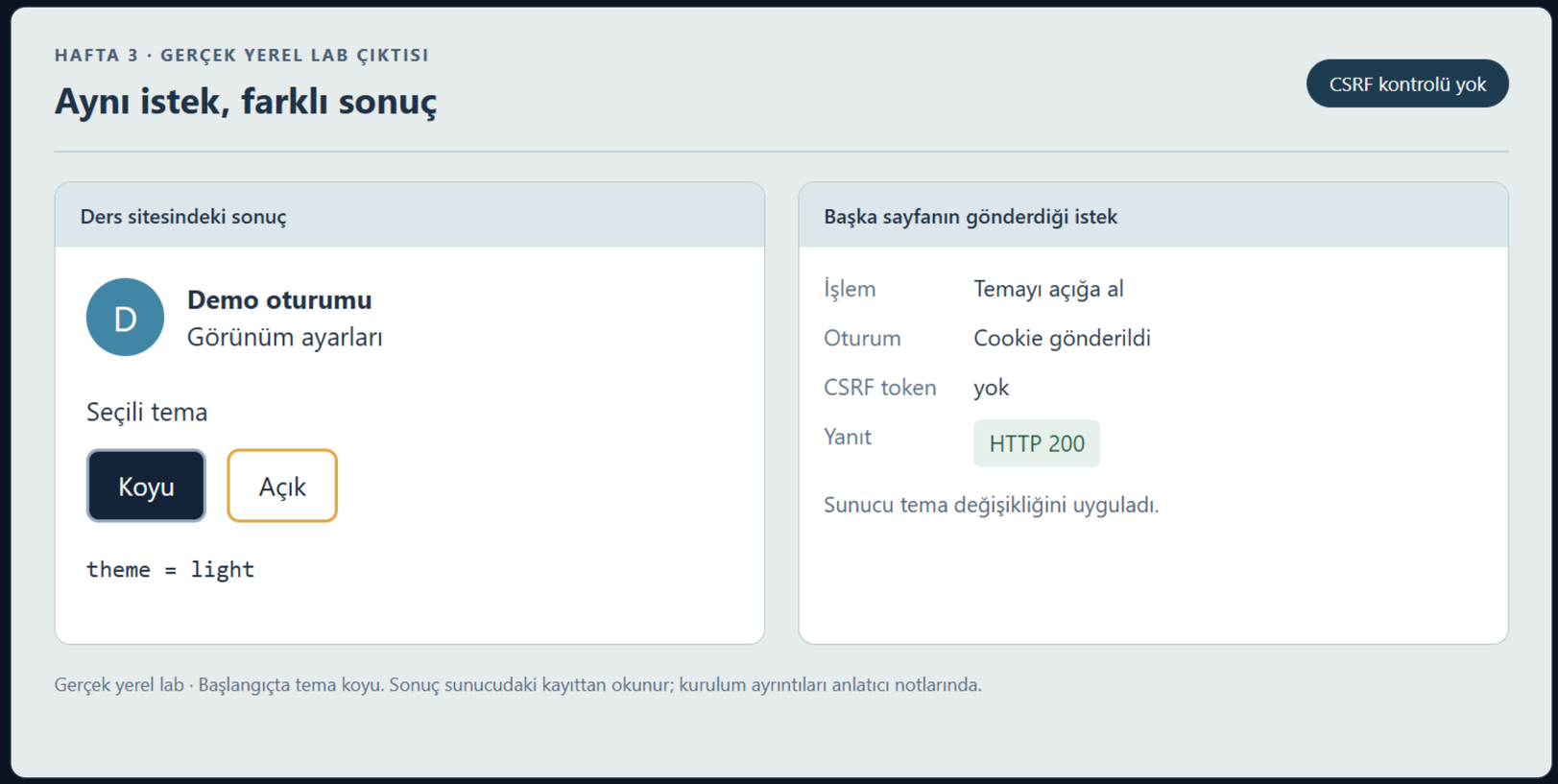
Sunucu

Cookie geçerli.
Oturum: Alice

Bu örnekte tarayıcı cookie'nin gönderilmesine izin veriyor.

Cookie oturumu tanıtır; işlemi Alice'in istediğini kanıtlamaz.

Kontrol yok: başka sayfa temayı değiştirdi



Gerçek lab: Alice ayar sayfasında işlem yapmadı; tema koyudan açığa döndü.

Cookie yetmesin; formun token’ı da eşleşsin



Token: sunucunun bu oturum için ürettiği tahmin edilemez değer.

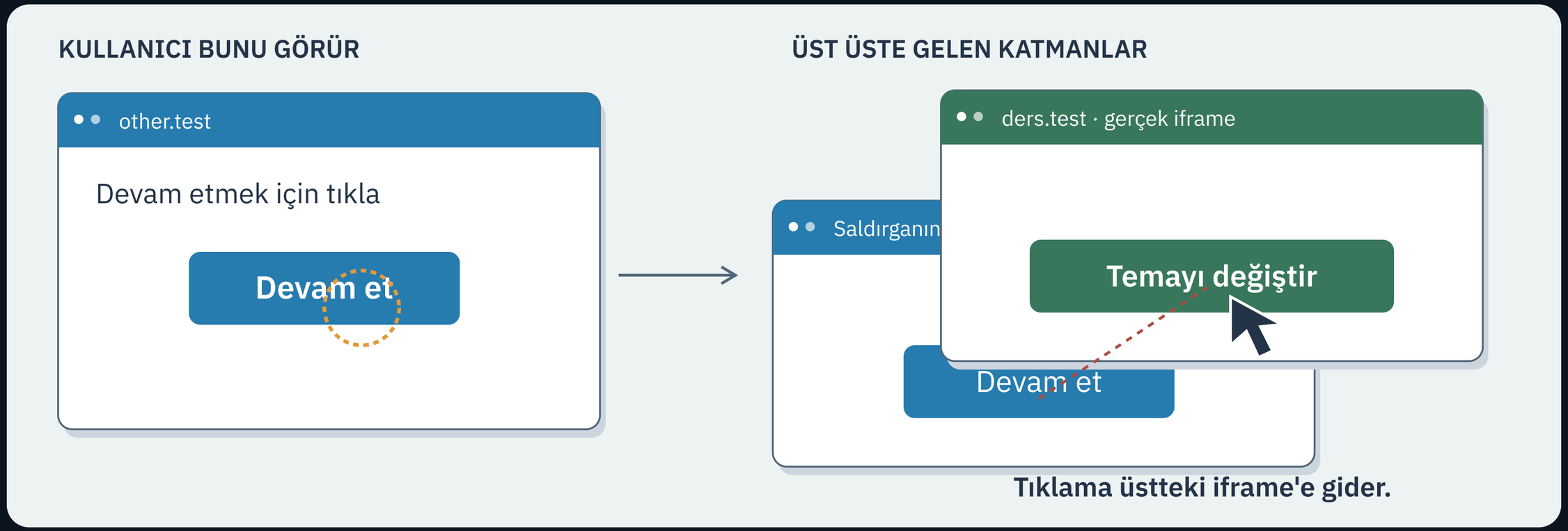
Aynı saldırı: istek reddedildi, tema değişmedi



Gerçek lab: token yok → HTTP 403. Tema koyu kaldı.

CLICKJACKING

Görünen düğme başka, tıklanan düğme başka



Saydam iframe'in gerçek düğmesi, görünen düğmenin üstüne hizalanır.

CLICKJACKING · SAVUNMA

Bu sayfa iframe içinde açılmasın

Hedef sayfanın response header'ları

```
Content-Security-Policy: frame-ancestors 'none'
X-Frame-Options: DENY
```

Doğrudan aç

İzin var

iframe içine koy

Tarayıcı engeller

CLICKJACKING · KONTROL

Aynı sayfayı iki şekilde aç

ders.test/settings

Tema ayarı

Doğrudan açılış: çalışır.

Şematik tarayıcı görünümü

other.test → iframe

İçerik engellendi

frame-ancestors 'none'

Şematik tarayıcı görünümü

TOPARLAYALIM

Üç saldırı, üç ana savunma

XSS

Kod çalıştırır

Sabit şablon
Autoescape

CSRF

İstek yaptırır

Oturuma bağlı token
Kaynak kontrolü

CLICKJACKING

Yanlış yere tıklatır

frame-ancestors

KAYNAKLAR · STANFORD CS253

Ders 3 için yararlandığımız bölümler

03 · Same Origin Policy

04 · Cookies and Sessions · 05 · Session attacks / CSRF

06 · XSS · 07 · XSS Defenses

08 · UI güvenliği bölümü

Feross Aboukhadijeh · Stanford · Fall 2021

KAYNAKLAR

Tarayıcı ve savunma referansları

MDN · Same-origin policy · CORS

OWASP · XSS Prevention · DOMPurify

OWASP · CSRF Prevention

OWASP · Clickjacking Defense · MDN · frame-ancestors

DEVTOOLS · YEREL LAB

Cookie ile storage farkını görelim

127.0.0.1:5023 · Console

```
1  document.cookie = "lesson_theme=dark; Path=/; SameSite=Lax";
2  localStorage.setItem("lesson_theme", "light");
3  sessionStorage.setItem("lesson_tab", "profile");
```

Application

Üç depoyu bul

→

Network

Run isteğini aç

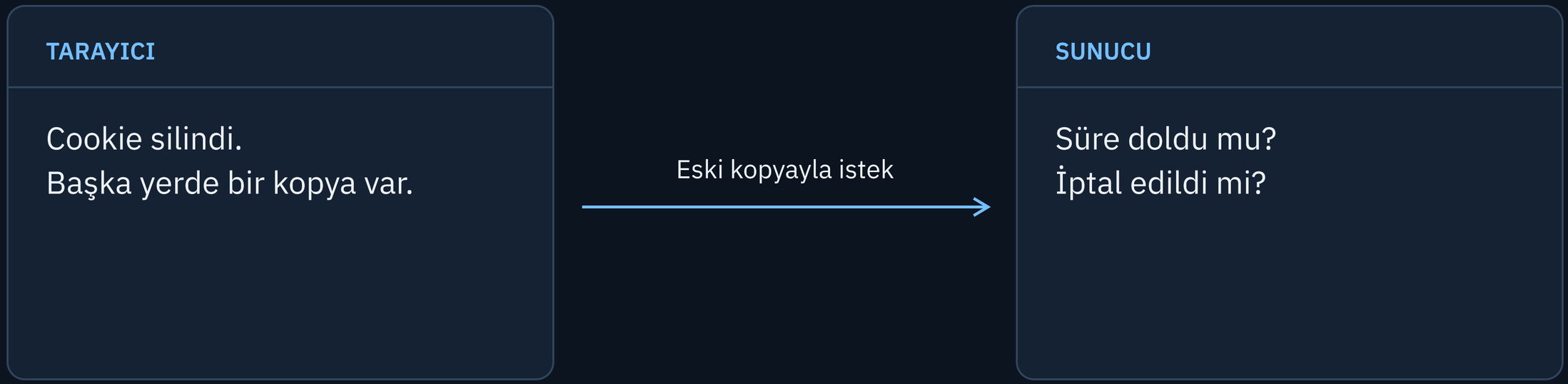
→

Cookie header'ı

Hangisi gönderilmiş?

KONUŞALIM

Çıkış yaptım. Eski cookie işe yarar mı?



KAYNAKLAR

HTTP, cookie ve session

MDN · HTTP · Set-Cookie

MDN · sessionStorage · localStorage

Flask · Sessions

Microsoft Learn · ASP.NET Core Session · Flask · Cookie ayarları

OWASP · Session Management

KAYNAKLAR

Çerez tercihleri ve tarayıcı ayarları

KVKK · 2023/1645 · Çerezler ve açık rıza

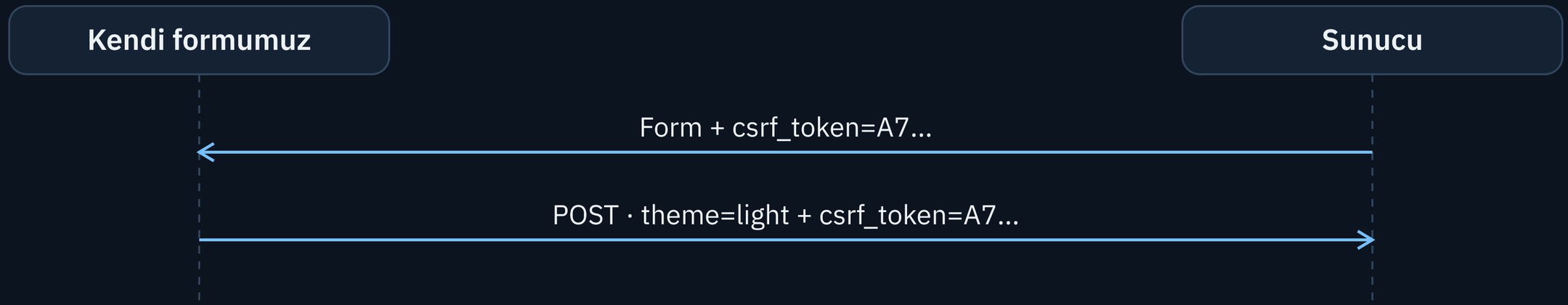
MDN · Third-party cookies

Slaytlardaki banner bir öğretim örneğidir; her amaç ayrıca değerlendirilir.

Kendi formumuza oturuma bağlı token ekle

Oturum için token oluştur

```
1  session["csrf_token"] = secrets.token_urlsafe(32)
```



EK · CSRF TOKEN DOĞRULAMA

Token eşleşmeden veriyi değiştirme

Sunucu · güncelleme kodundan önce

```
expected = session.get("csrf_token", "")
received = request.form.get("csrf_token", "")
if not expected or not secrets.compare_digest(
    expected.encode(), received.encode()
):
    abort(403)
```

Hazır framework korumasını kullan; bu kod kontrolün mantığı.

EK · TARAYICI MODELİ

Origin = scheme + host + port



SOP: Bir origin'deki script, diğer origin'in verisine serbestçe erişemez.

EK · ORIGIN ≠ SITE

İki alt alan adı aynı site olabilir

https://ders.example.test

Ders uygulaması

Origin 1

Şematik tarayıcı görünümü

https://api.example.test

API

Origin 2

Şematik tarayıcı görünümü

İkisi de aynı site: https + example.test

SameSite ve Origin ek kontrol sağlar

TARAYICI · SameSite=Lax

Cookie gönderimini sınırla

Cross-site POST'ta gönderilmez.

SUNUCU · Origin

Kaynağı doğrula

Beklenen origin ile tam eşleşme.

SameSite, CSRF token'ının genel yerine geçmez.